

Ejercicio 13

Parte 1 – Implementación de hashCode()

1. Clase Object

Todas las clases en Java heredan de la clase base `java.lang.Object`, que posee una implementación por defecto de `hashCode()`.

La documentación oficial establece que el valor retornado debe:

- Ser consistente durante la ejecución del programa.
- Retornar el mismo valor para objetos considerados iguales mediante `equals()`.
- No necesariamente ser único.

La implementación interna depende de la JVM, pero normalmente utiliza información relacionada con la identidad del objeto en memoria (dirección de memoria o una representación derivada de ella).

Ejemplo:

```
Object o1 = new Object();
Object o2 = new Object();

System.out.println(o1.hashCode());
System.out.println(o2.hashCode());
```

Cada objeto obtiene un hash diferente porque son instancias distintas.

2. Clase Integer

La clase `Integer` redefine `hashCode()` porque dos objetos `Integer` que contienen el mismo valor deben ser considerados equivalentes.

Implementación simplificada:

```
public int hashCode() {
    return Integer.hashCode(value);
}
```

```
}
```

```
public static int hashCode(int value) {  
    return value;  
}
```

Ejemplo:

```
Integer a = 10;  
Integer b = 10;  
  
System.out.println(a.hashCode()); // 10  
System.out.println(b.hashCode()); // 10
```

El hash es exactamente el valor entero almacenado.

3. Clase String

La clase String también redefine hashCode() porque dos cadenas con el mismo contenido deben producir el mismo hash.

Implementación:

```
public int hashCode() {  
    int h = hash;  
    if (h == 0 && value.length > 0) {  
        for (char c : value) {  
            h = 31 * h + c;  
        }  
        hash = h;  
    }  
    return h;  
}
```

Un ejemplo de como funciona esta formula matemática:

Ejemplo para "Hola":

H = 72

```
o = 111
l = 108
a = 97
hash = (((72 * 31 + 111) * 31 + 108) * 31 + 97)
```

```
hash = 2249042
```

¿Por qué las implementaciones son diferentes?

La diferencia surge porque cada clase posee una noción distinta de igualdad.

Object

La igualdad por defecto compara referencias:

```
o1 == o2
```

Por lo tanto el hash debe representar la identidad física del objeto.

Integer

La igualdad compara el valor numérico:

```
Integer a = 10;
Integer b = 10;

a.equals(b); // true
```

Dos objetos distintos deben producir el mismo hash si contienen el mismo entero.

String

La igualdad compara carácter por carácter:

```
"Hola".equals("Hola")
```

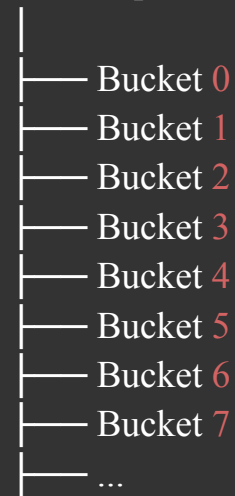
Por lo tanto el hash debe depender del contenido completo de la cadena.

En resumen:

Clase	Hash basado en
Object	Identidad del objeto
Integer	Valor entero
String	Secuencia de caracteres

Parte 2 – Estructura interna de HashMap

HashMap



Cada bucket es una posición de un arreglo interno.

Para insertar una clave:

1. Se calcula el hashCode().
2. Se obtiene el índice:

$\text{indice} = \text{hash} \& (\text{capacidad} - 1)$

3. Se inserta en dicho bucket.

Si dos claves generan el mismo índice ocurre una colisión.

Java utiliza:

- Lista enlazada para pocas colisiones.
- Árbol rojo-negro cuando hay muchas colisiones.

Hashes de las cadenas

Usando el algoritmo de String:

String	hashCode
Hola	2249042
HolaMundo	-801260770
HashMap	212340795
Colecciones	820767958

Capacidad inicial

Por defecto:

```
new HashMap<>()
```

crea una tabla de tamaño:

16 buckets

Índice:

$\text{indice} = \text{hash} \& 15$

Hola

$2249042 \& 15 = 2$

Bucket 2.

HolaMundo

$-801260770 \& 15 = 14$

Bucket 14.

HashMap

$212340795 \& 15 = 11$

Bucket 11.

Colecciones

$820767958 \& 15 = 6$

Bucket 6.

Estado del HashMap

Tabla interna (16 posiciones)

```
0 -> null
1 -> null
2 -> Hola
3 -> null
4 -> null
5 -> null
6 -> Colecciones
7 -> null
8 -> null
9 -> null
10 -> null
11 -> HashMap
12 -> null
13 -> null
14 -> HolaMundo
15 -> null
```